

AWS Certified AI Practitioner AIF-C01 One-Day Study Guide

What to learn, why it matters, how it works, and how the exam tests it

This guide takes you from zero to exam-ready in one focused day. Every topic follows the same pattern: **what** it is, **why** it matters (and how the exam uses it), and **how** it works or how to choose it. The exam is foundational: you will not write code or train models. You will recognise concepts, match scenarios to AWS services, and choose the safest, simplest, most cost-effective option.

Exam facts

Item	Detail
Exam code / level	AIF-C01, Foundational
Questions	65 total: 50 scored plus 15 unscored (unmarked, used by AWS to test future questions)
Passing score	700 on a scaled range of 100 to 1,000 (pass or fail)
Scoring model	Compensatory: you do not need to pass each domain, only the exam overall
Guessing	Unanswered = wrong, no penalty for wrong answers. Never leave a blank.
Question types	Multiple choice, multiple response, ordering, matching, case study
Duration and fee	Sources differ (90 minutes and USD 100 are commonly quoted). Confirm both on your booking page.
Validity	3 years, then recertify

Domain weights (share of scored questions)

Domain	Weight	Study time today
1. Fundamentals of AI and ML	20%	90 min
2. Fundamentals of generative AI	24%	90 min
3. Applications of foundation models	28%	120 min
4. Guidelines for responsible AI	14%	60 min
5. Security, compliance, and governance for AI solutions	14%	60 min

Contents

How to use this guide: the one-day plan	3
Domain 1: Fundamentals of AI and ML	5
Domain 2: Fundamentals of Generative AI	11
Domain 3: Applications of Foundation Models	15
Domain 4: Guidelines for Responsible AI	20
Domain 5: Security, Compliance, and Governance for AI Solutions	22
Look-alike services and concepts: compare and contrast	25
Master service picker and keyword decoder	26
One-page cheat sheet (read this before the exam)	28
Practice questions (30)	29
Answer key with explanations	33
Glossary and flashcards	35
Exam-day playbook	37

How to use this guide: the one-day plan

Why one day can work

AIF-C01 is a **recognition and judgement** exam, not a hands-on one. Most questions are short scenarios such as "a company wants X, which service or technique fits?" That means the winning skill is a solid mental map of *what each service is for*, plus a few concepts (RAG, fine-tuning, overfitting, responsible AI). A focused day can cover all five domains once. Retention comes from recall and practice questions, so the plan below alternates learning with testing yourself.

REMEMBER The three question types hiding inside every question

- **Definition:** "What is X?" (token, embedding, overfitting, hallucination).
- **Matching:** "Which AWS service fits this scenario?" (Textract, Rekognition, Bedrock, Clarify, Macie...).
- **Judgement:** "Which option is cheapest, simplest, most secure, or least effort?" AWS usually wants the managed, lowest-overhead answer that meets the requirement.

The study loop for every domain

1. **Read** the What and Why parts (10 to 15 minutes per topic). Do not memorise yet; understand.
2. **Trace** the How diagrams and numbered steps. Redraw one from memory on scrap paper.
3. **Recall:** close the guide and write the key tables from memory (service to purpose; technique to when to use it).
4. **Check:** do the Recall drill at the end of the domain, then re-read only what you missed.

DO THIS Study habits that raise your score

- Work in 50-minute blocks with 10-minute breaks. Stand up, drink water, no phone scrolling.
- Say answers out loud. If you cannot explain it simply, you do not know it yet.
- Keep a 'confusion list' of look-alike pairs (Textract vs Rekognition, fine-tuning vs RAG). the Look-alike section near the end covers the classics.
- Do not try to memorise every model name. Model lineups change; the exam tests concepts and service roles.

Hour-by-hour schedule (about 8.5 hours of focused study)

Time	Block	Goal by the end of the block
08:30 - 09:00	Orientation	Know the exam format, domain weights and this plan. Read this section.
09:00 - 10:30	Domain 1: AI and ML fundamentals	Explain learning types, overfitting, metrics, the ML lifecycle, and which pre-built AI service does what.
10:30 - 10:45	Break	Walk, water.
10:45 - 12:15	Domain 2: Generative AI fundamentals	Explain tokens, embeddings, transformers, temperature, hallucination, and the Bedrock/Q/JumpStart roles.
12:15 - 13:00	Lunch (no studying)	Rest your brain; memory consolidates during breaks.
13:00 - 15:00	Domain 3: Applications of foundation models	Master RAG, agents, prompt engineering, customization choices, and evaluation metrics. This is the biggest domain (28%).
15:00 - 15:15	Break	Walk, snack.

Time	Block	Goal by the end of the block
15:15 - 16:15	Domain 4: Responsible AI	Name the core dimensions and match Clarify, Guardrails, A2I, Model Cards to problems.
16:15 - 17:15	Domain 5: Security and governance	Apply shared responsibility; match IAM, KMS, PrivateLink, CloudTrail, Macie, Artifact, Config.
17:15 - 17:30	Break	
17:30 - 18:30	Practice questions	Do all 30 in 45 minutes without notes, then 15 minutes reviewing the answer key.
18:30 - 19:00	Look-alikes and cheat sheet	Read the Look-alike, Service picker and Cheat sheet sections to fix confusions.
19:00 - 19:30	Dinner	
19:30 - 20:15	Weak-spot repair	Redo missed questions, read the glossary aloud, re-read the exam traps.
20:15 - 20:30	Exam playbook	Read the Exam-day playbook, then sleep early. Sleep beats last-minute cramming.

EXAM TRAP Be honest about your baseline

- If you already know AWS and have used Bedrock or SageMaker, this day may be enough. If cloud and AI are both new, you may want a second pass or a second day. Use the practice questions as your gauge: **consistently 80 percent or higher** is a good sign you are ready.
- For extra practice, AWS Skill Builder offers an exam prep course and an official practice question set for AIF-C01. A 15-minute play in the Amazon Bedrock playground or PartyRock makes the concepts stick.

Domain 1: Fundamentals of AI and ML

Weight: **20%** | Study block: 90 minutes | Goal: speak the vocabulary fluently, choose the right ML approach, and match business needs to AWS AI services.

1.1 Core concepts: what the words mean

Why it matters: the exam builds wrong options out of terms that are *almost* right. Precise definitions let you eliminate them fast.

Term	What it is	Example
Artificial intelligence (AI)	Broad field: systems performing tasks that normally need human intelligence.	Chatbots, fraud alerts, route planning
Machine learning (ML)	Subset of AI. Algorithms learn patterns from data instead of following hand-written rules.	Predicting which customers will churn
Deep learning (DL)	Subset of ML using many-layered neural networks. Needs lots of data and compute.	Image recognition, speech, LLMs
Generative AI	Deep-learning models that create new content (text, images, code, audio).	Drafting an email, generating an image
Neural network	Layers of connected nodes with weights that are adjusted during training.	The engine of deep learning
NLP	AI that understands or generates human language.	Sentiment analysis, translation
Computer vision	AI that interprets images and video.	Defect detection, face matching
Algorithm vs model	Algorithm is the learning procedure. Model is the trained result that makes predictions.	Linear regression (algorithm) vs your trained price predictor (model)
Training vs inference	Training = learning from data. Inference = using the trained model on new data.	Fit the model, then call the endpoint
Feature vs label	Feature = input variable. Label = the correct answer to predict.	Features: age, income. Label: approved yes/no

REMEMBER The nesting to memorise

- AI contains ML, which contains deep learning, which contains generative AI. Exam distractors sometimes reverse this.

Data: what models learn from

- Labeled** data has the right answer attached (supervised learning). **Unlabeled** data does not.
- Structured** = tables with rows and columns. **Unstructured** = text, images, audio, video. **Semi-structured** = JSON, XML.
- Common shapes: tabular, time-series (values over time), text, image.
- Good data is **relevant, sufficient, representative, and clean**. Poor data gives poor models ("garbage in, garbage out") and is the main source of bias.

The four ways a model learns

Type	How it learns	Typical tasks	Everyday example
Supervised	From labeled examples: input plus correct answer	Classification (pick a category), regression (predict a number)	Spam vs not spam; house price

Type	How it learns	Typical tasks	Everyday example
Unsupervised	Finds structure in unlabeled data	Clustering, dimensionality reduction, anomaly detection	Customer segments
Reinforcement	Trial and error; rewards and penalties from an environment	Robotics, games, dynamic pricing	A robot learning to walk
Self-supervised	Creates its own labels from the data (predict the hidden part)	Pre-training of foundation models and LLMs	Predict the next word in a sentence

How a model learns (step by step)

1. **Split the data:** training set (learn), validation set (tune settings), test set (final, unbiased score). A common split is about 70/15/15 or 80/10/10.
2. The model makes a **prediction** from the features.
3. A **loss function** measures how wrong the prediction was.
4. An **optimizer** (for example gradient descent) nudges the weights to reduce the loss.
5. Repeat over many passes called **epochs**. **Hyperparameters** (learning rate, epochs, batch size) are chosen by you; **parameters** (weights) are learned.
6. Score on the **test set**. Deploy only if it meets the business goal.

WHY IT MATTERS Why keep a separate test set?

- A model tested on data it has already seen looks better than it really is. The untouched test set imitates the real world and exposes overfitting.

Fit: overfitting vs underfitting

	Overfitting	Underfitting	Good fit
Symptom	Great on training data, poor on new data	Poor on both training and new data	Good on both
Also called	High variance	High bias	Balanced
Cause	Model too complex, too little or too noisy data, trained too long	Model too simple, weak features, trained too little	Right complexity for the data
Fix	More data, regularization, simpler model, early stopping, dropout	More complex model, better features, train longer	Keep monitoring

Bias is error from assumptions that are too simple. **Variance** is sensitivity to noise in the training data. Lowering one usually raises the other, so the goal is balance.

Metrics: which question each one answers

Start from the **confusion matrix**: TP (true positive), FP (false positive), FN (false negative), TN (true negative).

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ overall correctness
Precision = $TP / (TP + FP)$ of the cases I flagged, how many were right?
Recall = $TP / (TP + FN)$ of all real cases, how many did I catch?
F1 score = $2 \times (Precision \times Recall) / (Precision + Recall)$

Metric	Use it when	Typical scenario
Accuracy	Classes are balanced and errors cost the same	Sorting photos of cats vs dogs
Precision	False positives are costly	Marking a real email as spam; flagging innocent customers
Recall (sensitivity)	False negatives are costly	Cancer screening, fraud detection, safety defects

Metric	Use it when	Typical scenario
F1 score	You need a balance of precision and recall, especially with imbalanced classes	Rare-event detection
AUC-ROC	Compare classifiers across all thresholds; higher is better (1.0 perfect, 0.5 random)	Model comparison
MAE, RMSE, R-squared	Regression (predicting numbers). MAE/RMSE: lower is better. R-squared: higher is better.	Price or demand forecasts
ROUGE / BLEU / BERTScore	Text generation quality (see Domain 3)	Summaries, translation

EXAM TRAP Accuracy can lie

- If 99 percent of transactions are legitimate, a model that always says "not fraud" is 99 percent accurate and completely useless. On imbalanced data use recall, precision, or F1.
- Memory hook: **Recall = Rescue every real case. Precision = Precisely right when I say yes.**

1.2 Practical use cases: when to use AI/ML (and when not)

Why it matters: the exam asks "is ML appropriate?" and "which kind of problem is this?" Getting the framing right picks the right service.

Use ML when...	Do not use ML when...
Rules are too many or too complex to write by hand	A simple rule or calculation solves it (tax = price x rate)
You have enough good historical data	You have little or poor-quality data
Patterns change and the system must adapt	You need 100 percent deterministic, guaranteed-exact results
You must personalise or scale decisions (millions of items)	The cost of building and running ML exceeds the benefit
Pattern-finding beats human speed (fraud, images, language)	You must fully explain each decision and cannot use an interpretable model

Match the business question to the ML problem type

Business question	ML problem type	Example
Which category does this belong to?	Classification	Spam or not spam; approve or reject a loan
How much or how many?	Regression	House price, energy demand
Are there natural groups?	Clustering (unsupervised)	Customer segmentation
Is this unusual?	Anomaly detection	Fraud, faulty sensor
What will this person want next?	Recommendation	Amazon Personalize
What happens over time?	Time-series forecasting	Next quarter's sales
Understand or produce language	NLP	Comprehend, Translate, LLMs
Understand pictures or video	Computer vision	Rekognition

How to choose the AWS layer: the ML stack

Layer	Services	What you manage	Choose when
AI services (pre-trained, called by API)	Rekognition, Textract, Comprehend, Transcribe, Polly, Translate, Lex, Kendra, Personalize	Almost nothing	The task is common; you want speed and no ML expertise
Generative AI platform	Amazon Bedrock (with Amazon Q apps on top)	Prompts, RAG data, optional customization	You need foundation models through an API
ML platform	Amazon SageMaker	Data, training, tuning, deployment	You need a custom model or full ML workflow control
Infrastructure and frameworks	EC2, Trainium, Inferentia, PyTorch, TensorFlow	Everything	Maximum control, specialist teams

REMEMBER The rule the exam loves

- **Pick the highest layer that solves the problem.** Phrases like "no ML expertise", "least operational overhead", or "fastest" point to pre-built AI services or Bedrock, not SageMaker.

Pre-built AI services: input to output

Service	Input	Output / what it does	Typical use
Amazon Rekognition	Images, video	Objects, faces, text in image, unsafe content	Content moderation, face verification
Amazon Textract	Scanned documents, PDFs	Text, forms (key-value), tables	Invoice and form processing
Amazon Comprehend	Text	Sentiment, entities, key phrases, language, topics, PII detection	Analysing reviews and tickets
Amazon Transcribe	Audio	Text transcript	Call-centre transcripts, captions
Amazon Polly	Text	Lifelike speech	Voice assistants, audiobooks
Amazon Translate	Text	Text in another language	Localisation
Amazon Lex	Text or voice	Conversational bot (intents and slots)	Customer-service chatbot
Amazon Kendra	Company documents	Intelligent enterprise search with natural-language answers	Internal knowledge search
Amazon Personalize	User interaction data	Personalised recommendations	Product and content suggestions
Amazon Augmented AI (A2I)	Low-confidence predictions	Routes them to human reviewers	Human-in-the-loop checks

1.3 The ML development lifecycle

Why it matters: questions ask where a step fits, which service does it, and what to do when a deployed model degrades.

Stage	What happens	Why	AWS tool
1. Define the goal	Turn a business problem into an ML question with a success metric	Otherwise you optimise the wrong thing	Stakeholder workshops

Stage	What happens	Why	AWS tool
2. Collect data	Gather data from databases, logs, files	Models are only as good as their data	Amazon S3, AWS Glue
3. Prepare data	Explore, clean, handle missing values, encode, engineer features	Usually the most time-consuming step	SageMaker Data Wrangler, Feature Store
4. Label data	Add correct answers where needed	Supervised learning needs labels	SageMaker Ground Truth
5. Train and tune	Fit the model, tune hyperparameters	Improve performance	SageMaker training, Autopilot
6. Evaluate	Score on the test set; check bias	Prove it meets the goal	SageMaker Clarify, metrics
7. Deploy	Serve predictions via an endpoint or batch job	Make it usable	SageMaker endpoints
8. Monitor	Watch quality, drift, cost	Models degrade over time	SageMaker Model Monitor, CloudWatch
9. Retrain	Update with fresh data	Keep accuracy up	SageMaker Pipelines

Data preparation vocabulary

- **Exploratory data analysis (EDA):** look at distributions and quality before modelling.
- **Cleaning:** handle missing values, duplicates, and outliers.
- **Feature engineering:** create better inputs (for example, day of week from a date). **Normalisation/scaling** puts features on similar ranges. **One-hot encoding** turns categories into numbers.

Inference types: how do you deliver predictions?

Type	Best for	Why
Real-time	Instant responses, small payloads (fraud check at checkout)	Low latency; endpoint always on
Serverless	Intermittent or spiky traffic	Scales to zero; pay only when used
Asynchronous	Large payloads or long processing (minutes)	Requests are queued
Batch	Score a big dataset offline (monthly churn scores)	Cheapest for non-urgent work

Model sources, drift, and MLOps

- **Pre-trained models** (for example from SageMaker JumpStart or open-source hubs) save time and cost versus training from scratch.
- **Data drift:** the input data changes (customers behave differently). **Concept drift:** the relationship between inputs and the right answer changes. Both lower accuracy; Model Monitor detects them.
- **MLOps** applies DevOps ideas to ML: repeatable pipelines, a model registry for versions, automated deployment and monitoring. Benefits: reproducibility, scalability, faster and safer releases.
- **Metrics come in two kinds:** technical (accuracy, F1, AUC) and business (cost per user, revenue, customer satisfaction, ROI). A model can be accurate and still fail the business goal.

The Amazon SageMaker family

Tool	What it does	Pick it when the question says...
SageMaker (AI)	End-to-end platform to build, train, deploy custom models	custom model, full control
Canvas	No-code ML with a visual interface	business analyst, no code

Tool	What it does	Pick it when the question says...
Autopilot	AutoML: tries algorithms and picks the best	automatically build and tune
JumpStart	Hub of pre-trained and foundation models, one-click deploy	start from a pre-trained model
Data Wrangler	Visual data preparation and cleaning	prepare or transform data
Feature Store	Central store for reusable features	share features across models
Ground Truth	Data labeling with human workers	label training data
Clarify	Detect bias, explain predictions, evaluate FMs	bias or explainability
Model Monitor	Detect drift and quality issues in production	model degrades over time
Model Cards	Documentation of a model's purpose, data, metrics, risks	governance documentation
Model Registry / Pipelines	Version models; automate ML workflows	MLOps, repeatable workflow

EXAM TRAP Domain 1 traps

- Supervised = labeled; unsupervised = unlabeled. Clustering is **unsupervised**, classification is **supervised**.
- High training score but low test score = **overfitting**, not underfitting.
- Rekognition reads *pictures*; Textract reads *documents* (forms, tables). Comprehend analyses *text meaning*; Kendra *searches* documents.
- Ground Truth = labeling. A2I = human review of predictions. Do not swap them.

DO THIS Recall drill (cover the page, answer aloud)

- Define AI, ML, DL, generative AI in one line each and show how they nest.
- Which learning type is clustering? Which is next-word prediction in an LLM?
- Write the formulas for precision and recall. Which do you maximise for cancer screening?
- Name three ways to fix overfitting.
- List the nine lifecycle stages and one AWS tool for four of them.
- Which service transcribes speech? Reads invoices? Recommends products? Flags unsafe images?

Domain 2: Fundamentals of Generative AI

Weight: **24%** | Study block: 90 minutes | Goal: explain how LLMs work, control them with inference parameters, and state clearly what generative AI is good and bad at.

2.1 Core concepts of generative AI

Why it matters: these terms appear in almost every later question, in Domain 3 especially. Learn them properly once.

Term	What it is	Why the exam cares
Foundation model (FM)	Very large model pre-trained on broad, mostly unlabeled data (self-supervised); adaptable to many tasks	Reuse instead of training from scratch
Large language model (LLM)	An FM specialised in language; predicts the next token	Chat, summaries, code
Token	A chunk of text (a word or part of a word) the model reads and writes	Limits and pricing are counted in tokens
Embedding (vector)	List of numbers that captures meaning; similar meaning = nearby vectors	Powers semantic search and RAG
Transformer	Architecture using self-attention to relate every token to every other token	The design behind modern LLMs
Context window	Maximum tokens the model handles per request (prompt plus response)	Very long documents may not fit
Prompt	The instruction and context you send to the model	Main way to steer output
Multimodal model	Handles more than one data type (text, image, audio, video)	Describe an image, answer questions about a chart
Diffusion model	Generates images by learning to remove noise step by step	Text-to-image tools
GAN	Two networks compete: a generator and a discriminator	Older image generation approach
VAE	Encodes data into a compact latent space, then decodes new samples	Generation and compression
Hallucination	Fluent but false or invented output	Main GenAI risk; RAG and Guardrails reduce it

How an LLM produces an answer

1. Your prompt is split into **tokens**.
2. Each token becomes an **embedding** (a vector).
3. Transformer layers use **self-attention** to work out which tokens relate to each other.
4. The model outputs a **probability for every possible next token**.
5. One token is chosen (guided by temperature, Top P, Top K) and added to the text. Steps 3 to 5 repeat until a stop condition or the maximum length.

WHY IT MATTERS Why hallucinations happen

- The model predicts *plausible* text, not *verified* facts. It has no built-in fact checker, and its knowledge stops at its training cutoff. This is why grounding the answer in your own data (RAG) is so important.

How embeddings enable search

An embeddings model converts text into a vector. Texts with similar meaning land close together, even if they share no words ("car" and "automobile"). To search, embed the question and find the nearest vectors by similarity (for example cosine similarity). This is **semantic search**, and it is the retrieval engine inside RAG.

Inference parameters: your control knobs

Parameter	What it does	Lower or off	Higher
Temperature	Controls randomness in token choice	Focused, repeatable, factual: Q&A, extraction, classification	Creative, varied: brainstorming, story writing
Top P	Sample only from the smallest set of tokens whose probabilities add up to P	Safer, more predictable wording	More diverse wording
Top K	Sample only from the K most likely tokens	More predictable	More varied
Max tokens (response length)	Caps output length	Shorter, cheaper answers	Longer answers, higher cost
Stop sequences	Text that makes the model stop generating	-	-

EXAM TRAP Temperature direction

- "Make answers more consistent / less random" means **lower** the temperature. "More creative" means raise it. Temperature changes randomness, not the model's knowledge.

Generative AI use cases

Use case	Example
Summarisation	Condense a 40-page report or a call transcript
Conversational assistants	Customer-support chatbot, internal help desk
Content creation	Marketing copy, product descriptions, emails
Code generation and explanation	Amazon Q Developer, code review, test generation
Translation and rewriting	Adjust tone, simplify language
Search and Q&A over documents	RAG assistants, Amazon Q Business
Image, audio, video generation	Product images, voiceovers
Data extraction and analysis	Pull fields from text, classify sentiment
Personalisation	Tailored recommendations and messages

The foundation model lifecycle

1. **Data selection:** choose and prepare large, diverse, high-quality data.
2. **Model selection:** choose an architecture or an existing FM that fits the task, cost and constraints.
3. **Pre-training:** learn general language or image patterns from massive unlabeled data (very expensive).
4. **Fine-tuning or adaptation:** specialise for a task or domain with smaller targeted data.
5. **Evaluation:** test with metrics, benchmarks and human review.
6. **Deployment:** serve the model (for example through Bedrock).
7. **Feedback and monitoring:** collect user feedback, watch quality and safety, improve.

2.2 Capabilities and limitations for business problems

Why it matters: scenario questions ask whether generative AI fits a problem and how to reduce its weaknesses.

Advantages	Limitations and risks
Adaptable: one model handles many tasks	Hallucinations: confident but wrong output
Fast prototyping and lower barrier to entry	Nondeterministic: the same prompt can give different answers
Boosts productivity and creativity	Bias and toxicity inherited from training data
Responsive, personalised experiences at scale	Privacy and IP risks if sensitive data is used carelessly
Works on unstructured data (text, images)	Limited interpretability (a black box)
Simplifies many workflows (summarising, drafting)	Cost and latency at scale; knowledge cutoff

Limitation to mitigation

Problem	How to reduce it
Hallucination	RAG (ground in your data), lower temperature, instruct "answer only from the context", Guardrails grounding checks, human review
Out-of-date knowledge	RAG with a current knowledge base; models cannot know events after their training cutoff
Inconsistent answers	Lower temperature, clear prompt templates, few-shot examples, testing
Bias or toxic content	Guardrails content filters, diverse and balanced data, evaluation, human oversight
Privacy leaks	Keep PII out of prompts or redact it (Guardrails), encryption, private endpoints, IAM least privilege
High cost or latency	Choose a smaller model, shorten prompts and outputs, batch mode, right pricing model

How to choose a foundation model

- **Task and modality fit:** text, image, multimodal, embeddings.
- **Quality** on your task (test with your own examples), **latency**, and **cost per token**.
- **Context window size** for long documents; **language support**.
- **Customisation options** (fine-tuning supported?), **licensing** and provider data policies, **compliance** needs, and **Region** availability.
- Bigger is not always better: smaller models are cheaper and faster, and are often enough.

Business metrics for GenAI

- Cost per user or per request, and overall **ROI**.
- Conversion rate, average revenue per user, customer lifetime value.
- Efficiency and productivity gains (time saved, tickets resolved), user satisfaction, response time.

2.3 AWS infrastructure and services for generative AI

Service	What it is	Choose it when
Amazon Bedrock	Fully managed, serverless API access to foundation models from several providers, with RAG, agents, guardrails and evaluation built in	You want to use or lightly customise FMs without managing servers. The default answer for most GenAI app questions.
SageMaker JumpStart	Model hub inside SageMaker; deploy and fine-tune FMs on infrastructure you control	You want more control over hosting and training, or an open model not offered in Bedrock

Service	What it is	Choose it when
Amazon Q Business	GenAI assistant over your company data through connectors; respects existing user permissions	Employees need answers from internal documents and apps
Amazon Q Developer	Coding assistant and AWS help	Developers want code generation, explanation, debugging
PartyRock	No-code playground for building small GenAI apps (built on Bedrock)	Learning and quick, low-stakes experiments
AWS Trainium and Inferentia	AWS-designed chips for cheaper ML training and inference	Cost-efficient training or inference at scale

Why build on AWS?

- **Accessibility and speed to market:** managed APIs remove infrastructure work.
- **Cost-effectiveness:** pay per use (per token) or reserve capacity when traffic is steady.
- **Security and compliance:** encryption, IAM, private connectivity, and compliance certifications.
- **Scalability and choice:** many models behind one API; switch models without rebuilding.
- **Responsible-AI tooling:** Guardrails, evaluation, Clarify, A2I.

EXAM TRAP Domain 2 traps

- Bedrock = use FMs via API (managed). SageMaker = build, train, host your own. Amazon Q = ready-made assistant.
- Temperature controls randomness. Context window controls input plus output length. Do not confuse them.
- Output tokens are usually priced higher than input tokens; cost scales with tokens, not with number of users alone.
- A foundation model is pre-trained on **unlabeled** data with self-supervised learning.

DO THIS Recall drill

- Explain in five steps how an LLM generates a reply.
- What does an embedding represent, and why does it enable semantic search?
- Which setting would you change for a legal Q&A bot? For a poetry generator?
- Give three GenAI limitations and one mitigation for each.
- Bedrock vs JumpStart vs Q Business vs PartyRock: one sentence each.

Domain 3: Applications of Foundation Models

Weight: **28%** (the biggest domain) | Study block: 120 minutes | Goal: design a GenAI application, write good prompts, choose the right customisation method, and evaluate the results.

3.1 Design considerations for FM applications

Amazon Bedrock in one page

What: a fully managed, serverless service that gives API access to foundation models from several providers (for example Anthropic, Amazon, Meta, Mistral AI, Cohere, AI21 Labs, Stability AI; the lineup changes, so learn the concept, not the list). **Why:** you avoid managing infrastructure, can try different models behind one API, and your prompts and data are not used to train the base models and stay encrypted in your account. **How:** pick a model, send a prompt (or use Knowledge Bases, Agents, Guardrails on top), pay per use.

Bedrock feature	What it does	Use it when
Playgrounds	Try models and prompts in the console	Quick experiments
Knowledge Bases	Managed RAG: ingests your data, chunks, embeds, stores vectors, retrieves at query time	Answers must come from your documents
Agents	FM plans a multi-step task and calls your APIs and knowledge bases	The app must take actions, not just chat
Guardrails	Content filters, denied topics, word filters, PII redaction, grounding checks	Safety, compliance, brand control
Model evaluation	Compare models with automatic metrics, human review, or LLM-as-a-judge	Choose the best model for your task
Customisation	Fine-tuning and continued pre-training on your data	A model must learn your task, style or domain
Invocation logging	Record prompts and responses to S3 or CloudWatch Logs	Audit, monitoring, troubleshooting

Pricing mode	How it works	Choose it when
On-demand	Pay per input and output token; no commitment	Getting started, unpredictable or low traffic
Batch	Submit many prompts as a job; cheaper; results later	Large offline workloads, no need for instant reply
Provisioned Throughput	Reserve model capacity for a fixed term; consistent performance	Steady high traffic. Also required to serve a custom (fine-tuned) model

RAG: Retrieval-Augmented Generation

What: before answering, the app retrieves relevant passages from your own data and adds them to the prompt, so the FM answers from current, private facts. **Why:** models do not know your internal documents, their knowledge is dated, and they hallucinate; retraining for every change is slow and costly. RAG fixes all three *without changing the model*.

How it works

1. **Ingest (offline):** split documents into chunks, turn each chunk into an embedding with an embeddings model (for example Amazon Titan Embeddings or Cohere Embed), store the vectors in a vector database.

2. **Ask (online):** the user's question is embedded with the same model.
3. **Retrieve:** similarity search returns the top matching chunks.
4. **Augment:** chunks are inserted into the prompt with instructions such as "answer only using this context".
5. **Generate:** the FM writes a grounded answer, ideally with source citations.

Vector store on AWS	Note
Amazon OpenSearch Service / Serverless	Popular default for vector search; native Bedrock Knowledge Bases option
Amazon Aurora PostgreSQL and Amazon RDS for PostgreSQL (pgvector)	Use when you already run PostgreSQL
Amazon DocumentDB, Amazon Neptune, Amazon MemoryDB	Also support vector search for their data models
Third-party (Pinecone, Redis Enterprise Cloud, MongoDB Atlas)	Supported by Knowledge Bases

REMEMBER RAG vs fine-tuning at a glance

- RAG adds **knowledge** at question time. Fine-tuning changes **behaviour or style** in the model's weights.
- Data changes often, needs citations, or is confidential: **RAG**. Consistent tone, format, or a specialised task with labeled examples: **fine-tuning**.
- RAG is usually cheaper and faster to start, and updating is as easy as updating the documents.

Agents: when the model must act

An FM alone can only produce text. **Agents for Amazon Bedrock** let it complete multi-step tasks.

1. The user asks for something ("cancel order 123 and refund me").
2. The agent (FM) **reasons and plans** the steps.
3. It calls **action groups** (your APIs, often via AWS Lambda) to fetch data or perform actions.
4. It queries a **knowledge base** when it needs facts.
5. It observes results, repeats if needed, and returns the final answer.

3.2 Effective prompt engineering

What: designing the input so the model returns what you want. **Why:** it is the cheapest, fastest way to improve output. No training, no infrastructure, instant to iterate. **How:** follow the elements and techniques below.

Element of a good prompt	Purpose	Example
Instruction	The task to do	Summarise the text in three bullet points
Context	Background that narrows the answer	You are a support agent for an airline
Input data	The content to work on	[customer email text]
Output indicator	Format and length	Reply in JSON with keys: sentiment, reason

Weak prompt: Tell me about our returns.

Strong prompt: You are a retail support agent. Using **ONLY** the policy below, answer the customer's question in under 80 words. If the policy does not cover it, say you do not know. Policy: [...] Question: [...]

Technique	What it is	Use it when
Zero-shot	Instruction only, no examples	Simple, common tasks

Technique	What it is	Use it when
One-shot / few-shot	Include one or several worked examples in the prompt	You need a specific format or style, or the task is unusual
Chain-of-thought	Ask the model to reason step by step	Math, logic, multi-step reasoning
Prompt template	Reusable prompt with placeholders	Consistency across many requests
Role / persona	Tell the model who to be	Tone and domain focus
Negative prompt	State what to avoid	Exclude unwanted content or image features

Best practices

- Be specific and concise; state the audience, length, tone and format.
- Give examples of the desired output (few-shot).
- Separate instructions from data with delimiters (for example triple quotes or XML tags).
- Tell the model what to do when it does not know ("say you do not know") to reduce hallucination.
- Iterate: test, look at failures, refine. Try different temperatures.

Prompt attacks (risks and defences)

Threat	What it is	Defence
Prompt injection	Hidden instructions in user input or retrieved content that override yours	Guardrails, input validation and filtering, separate trusted vs untrusted text, least-privilege tools
Jailbreaking	Tricking the model into ignoring its safety rules	Guardrails, safety testing (red teaming)
Prompt leaking / exposure	Extracting your system prompt or private data	Do not put secrets in prompts; output filtering
Hijacking	Redirecting the model to serve the attacker's goal	Guardrails, restrict what actions the agent can take
Poisoning	Corrupting training or retrieval data	Data validation, access control, trusted data sources

3.3 Training and fine-tuning foundation models

What: ways to make a general FM better for your needs. **Why:** the more you change the model, the higher the cost, time, data and expertise required, so you should climb the ladder one rung at a time. **How:** start at the top and stop when quality is good enough.

Approach	What changes	Data needed	Cost and effort	Use when
1. Prompt engineering	Nothing in the model	None (maybe a few examples)	Lowest	Always try first
2. RAG	Nothing in the model; adds context at run time	Your documents (any format)	Low	Fresh or private knowledge, citations
3. Fine-tuning	Weights adjusted for a task or style	Labeled examples (prompt and desired completion)	Medium	Consistent format, tone or specialised task

Approach	What changes	Data needed	Cost and effort	Use when
4. Continued pre-training	Weights updated to learn a domain	Large unlabeled domain text	High	Teach domain vocabulary and knowledge
5. Pre-train from scratch	Everything	Massive datasets	Extreme	Rarely justified

Related terms

- **Instruction tuning:** fine-tune on instruction and response pairs so the model follows directions better.
- **RLHF (reinforcement learning from human feedback):** humans rank outputs; the model learns to prefer helpful, safe answers.
- **Transfer learning:** reuse a model trained on one task as the starting point for another.
- **Distillation:** train a small model to imitate a large one for cheaper, faster inference.
- **Domain adaptation:** adjust a model to a specific field such as law or medicine.

Preparing data for fine-tuning

- **Curate:** accurate, clean, relevant examples. Quality matters more than quantity.
- **Label:** correct prompt and response pairs (SageMaker Ground Truth can help).
- **Representative and balanced:** cover the real range of inputs and avoid bias.
- **Governance:** check privacy, licensing and permission to use the data; remove PII.
- **Size:** enough examples for the task; more is not always better.

EXAM TRAP Fine-tuning traps

- Fine-tuning needs **labeled** data. Continued pre-training uses **unlabeled** data.
- "Cheapest way to improve" is prompt engineering. "Add latest company knowledge" is RAG. Neither retrains the model.
- A fine-tuned or custom model on Bedrock needs **Provisioned Throughput** to be served.
- Do not fine-tune to add facts that change often; use RAG.

HOW IT WORKS Quick decision guide

- "Answer from our documents / up-to-date" -> RAG.
- "Speak in our brand voice / always output this format" -> few-shot prompts first, then fine-tuning.
- "Model does not understand our industry jargon; we have a big pile of unlabeled text" -> continued pre-training.
- "Perform actions in our systems" -> Agents.
- "Block harmful topics" -> Guardrails.

3.4 Evaluating foundation model performance

What: measuring how good the output is. **Why:** generative output is open-ended, so 'accuracy' alone does not work; you also need to know if it meets the business goal. **How:** combine automatic metrics, benchmark datasets, human judgement and business KPIs.

Metric	What it measures	Used for
ROUGE	Overlap of words and phrases (n-grams) between generated text and a reference; ROUGE-N, ROUGE-L	Summarisation
BLEU	N-gram overlap with reference translations (precision focused)	Machine translation

Metric	What it measures	Used for
BERTScore	Semantic similarity using embeddings, catches same meaning in different words	Text generation quality
Perplexity	How well the model predicts text; lower is better	Language-model quality
Accuracy, F1	Correctness on labeled tasks	Classification-style GenAI tasks

Evaluation approach	Good for	Limitation
Automatic metrics	Fast, cheap, repeatable comparisons	Miss nuance, tone, factual truth
Benchmark datasets (for example GLUE, SuperGLUE, SQuAD, MMLU, HELM)	Standard comparison across models	May not match your use case
Human evaluation	Helpfulness, tone, safety, edge cases	Slow and costly
LLM-as-a-judge	Scaling quality scoring with a judge model	Judge can be biased; validate with humans

How to evaluate step by step

1. Define what 'good' means for the business (accuracy, tone, speed, cost).
2. Build a test set of real prompts with reference answers.
3. Run candidate models (Amazon Bedrock model evaluation can do this).
4. Score with metrics, human raters, or a judge model.
5. Compare quality, latency and cost together, then re-test after every change.

EXAM TRAP Domain 3 traps

- ROUGE = summaries. BLEU = translation. BERTScore = meaning similarity. AUC/RMSE are classic ML, not text generation.
- RAG needs an **embeddings model** plus a **vector store**; the FM alone is not enough.
- Guardrails filter and constrain; they do not add knowledge. Knowledge Bases add knowledge; they do not block topics.
- Agents = actions and multi-step tasks. Knowledge Bases = retrieval. They can work together.

DO THIS Recall drill

- Draw the RAG flow (ingest and query) from memory.
- Fill in the customisation ladder: five rungs, data needed for each, cost trend.
- Write one weak and one strong prompt for the same task; name the techniques used.
- Which metric for a summary? For a translation? For similarity of meaning?
- Which Bedrock pricing mode serves a fine-tuned model?

Domain 4: Guidelines for Responsible AI

Weight: **14%** | Study block: 60 minutes | Goal: describe what makes AI responsible, spot bias and risk, and match AWS tools to responsibility problems.

4.1 Developing responsible AI systems

What: building and using AI so it is fair, safe, explainable, private, and accountable. **Why:** biased or unsafe AI harms people, breaks laws, and destroys customer trust. **How:** design for the dimensions below and use tools to check them throughout the lifecycle.

Core dimension	What it means	Example / tool
Fairness	Outputs do not systematically disadvantage groups	Clarify bias metrics; balanced datasets
Explainability	You can explain why the model produced an output	Clarify (SHAP feature attributions)
Privacy and security	Protect personal data and systems	Encryption, IAM, PII redaction, Guardrails
Safety	Systems do not cause harm to people or society	Guardrails content filters, red teaming
Controllability	Humans can steer, override, or stop the system	Human-in-the-loop with A2I
Veracity and robustness	Correct outputs and stable behaviour, even with unexpected input	Evaluation, RAG, monitoring
Governance	Policies, roles, and processes ensure accountability	Model Cards, audits, documentation
Transparency	Stakeholders know how the system works and its limits	Model Cards, AWS AI Service Cards, disclosure

Where bias comes from and what it does

- **Sources:** unrepresentative or skewed training data, biased labels, missing groups, historic prejudice in the data, sampling errors.
- **Effects:** unfair decisions for demographic groups, inaccurate results (underfitting, high bias), models that memorise noise (overfitting, high variance), and loss of trust.
- **Data characteristics that help:** diversity, inclusivity, balanced and curated data sources, and regular re-checks.

Risks specific to generative AI

- **Hallucination** and toxic or offensive output.
- **Intellectual property and copyright** exposure; legal risk from generated content.
- **Privacy leaks** of personal or confidential data; **loss of customer trust** and misuse.
- **Mitigate** with guardrails, grounding (RAG), human oversight, testing, clear disclosure that content is AI-generated.

Choosing models responsibly

- Match the model to the use case and its risk level (a medical or lending decision needs more care than a marketing draft).
- Consider **environmental impact** and sustainability (smaller models use less energy), **licensing and provenance**, and business risk.
- Keep humans involved for high-stakes decisions.

AWS tools for responsible AI

Tool	Responsible-AI job	When the question says...
SageMaker Clarify	Detect bias in data and models; explain predictions; evaluate FMs	bias, fairness, feature importance
Guardrails for Amazon Bedrock	Filter harmful content, block topics, redact PII, check grounding	block toxic or off-topic output
Amazon A2I	Human review of low-confidence predictions	human in the loop
SageMaker Model Monitor	Detect data or quality drift after deployment	model degrades over time
SageMaker Model Cards	Document intended use, training data, metrics, risks	document the model for governance
AWS AI Service Cards	AWS's transparency docs for its AI services	intended use and limits of an AWS service
Bedrock model evaluation	Assess quality, safety and responsibility metrics	compare models responsibly

4.2 Transparent and explainable models

What: Transparency is openness about how a system was built, trained and is used. **Explainability** is being able to say why a model made a specific decision. **Why:** regulators, customers and auditors demand reasons (for example why a loan was denied), and explanations expose bias. **How:** use interpretable models where possible, add explanation tools, and document everything.

	Transparent / interpretable models	Black-box models
Examples	Linear regression, decision trees	Deep neural networks, LLMs
Strength	Easy to understand and audit	Often higher performance on complex data
Weakness	May be less accurate on complex tasks	Hard to explain without extra tools

- **Performance vs interpretability trade-off:** the more complex the model, the harder it is to explain. Pick according to risk: regulated, high-impact decisions lean towards explainability.
- **Tools:** Clarify (SHAP values show how much each feature pushed the prediction), partial dependence plots, Model Cards, AI Service Cards, documented **data lineage** and sources.
- **Human-centred design:** involve users and stakeholders so explanations are understandable, not just technical.

EXAM TRAP Domain 4 traps

- Clarify = bias and explainability. Guardrails = content safety for GenAI. A2I = humans reviewing. Model Cards = documentation. Do not mix these up.
- Bias is mostly a **data** problem. Fix it with better, more representative data and audits, not just a bigger model.
- Higher accuracy does not automatically mean better. In regulated fields, explainability may win.

DO THIS Recall drill

- Name the eight core dimensions of responsible AI and one tool or example for each.
- Give two sources of bias and two mitigations.
- Why can a decision tree be more suitable than a deep network for loan approval?
- Match: bias detection, harmful text filtering, human review, model documentation.

Domain 5: Security, Compliance, and Governance for AI Solutions

Weight: **14%** | Study block: 60 minutes | Goal: know who secures what, protect AI data and access, and match governance and compliance services to needs.

5.1 Securing AI systems

Shared responsibility model

What: AWS and you split security duties. **Why:** exam questions ask who is responsible for what. **How to decide:** if it is the physical or managed infrastructure, it is AWS; if it is your data, access, and how you use the service, it is you.

AWS is responsible for (security OF the cloud)	You are responsible for (security IN the cloud)
Data centres, hardware, networking, the managed service infrastructure	Your data and what you put in prompts and training sets
Securing the underlying Bedrock and SageMaker services	IAM users, roles and permissions (least privilege)
Compliance certifications of the platform	Encryption choices, key management, logging, application security, model use policies

Core security controls

Control	What it does	AI example
IAM (roles and policies)	Controls who can call what. Apply least privilege ; prefer roles over long-term keys	Only the app role can invoke a specific Bedrock model
AWS KMS and TLS	Encrypt data at rest (keys you manage) and in transit	Encrypt training data in S3 and fine-tuned custom models
VPC endpoints (AWS PrivateLink)	Private connection to AWS services; traffic stays off the public internet	Call Bedrock from a private VPC
Amazon S3 controls	Bucket policies, block public access, versioning	Secure training and RAG documents
AWS Secrets Manager	Store and rotate credentials	API keys for tools an agent uses
Amazon Macie	Discovers sensitive data (PII) in S3	Scan a dataset before fine-tuning
Amazon Inspector	Vulnerability scanning of workloads	Scan compute that hosts your model app
AWS CloudTrail	Audit log of API calls (who did what, when)	Who invoked which model
Amazon CloudWatch	Metrics, logs, alarms	Watch latency, errors, token usage
Bedrock Guardrails	Runtime safety controls on inputs and outputs	Redact PII, block topics

Protecting AI data and models (best practices)

- Assess **data quality** and protect **integrity** (prevent poisoning); document **data lineage** and sources.
- Use **privacy-enhancing techniques**: anonymisation, masking, minimising data collected, encryption.
- Control access with **least privilege**; log and monitor usage; keep secure **data engineering** pipelines.
- Test for prompt attacks and adversarial input; add input and output filtering.
- Keep data in approved **Regions** for data residency requirements.

Generative AI Security Scoping Matrix

What: AWS's framework that shows how much security responsibility you carry depending on how you use GenAI. **Why:** the more control you take, the more you own. **How:** the scopes run from least to most ownership:

Scope	How you use GenAI	You mainly own
1. Consumer app	Use a public GenAI app (for example a chatbot website)	Usage policies and what data staff enter
2. Enterprise app	Use a SaaS product with GenAI features (for example Amazon Q)	Data permissions, configuration, usage
3. Pre-trained model	Build your app on an existing FM (for example through Bedrock)	App security, prompts, RAG data, access control
4. Fine-tuned model	Customise an FM with your data	The above plus training-data protection and the custom model
5. Self-trained model	Build and train your own model from scratch	Everything, including training data and model security

5.2 Governance and compliance for AI

What: the policies, standards, and evidence that show your AI meets laws and internal rules. **Why:** regulated industries and auditors require proof, and AI adds new obligations (fairness, transparency, data handling). **How:** combine governance processes with AWS services that collect and prove compliance.

AWS service	Governance / compliance job
AWS Artifact	On-demand access to AWS compliance reports (SOC, ISO and others) and agreements
AWS Audit Manager	Automates evidence collection and audit-ready reports, including for AI workloads
AWS Config	Tracks resource configurations and checks them against rules
AWS CloudTrail	Records API activity for auditing and investigations
AWS Trusted Advisor	Best-practice checks (security, cost, limits)
Amazon Macie / Inspector	Data-sensitivity discovery / vulnerability scanning
SageMaker Model Cards	Records model purpose, data, evaluation and risks for governance

Regulatory and compliance standards

- **Standards and frameworks:** ISO (for example ISO 27001 for security, ISO/IEC 42001 for AI management), SOC reports, and the NIST AI Risk Management Framework.
- **Laws and regulations:** privacy laws (GDPR, HIPAA for health data) and emerging AI rules such as risk-based AI regulation (for example the EU AI Act) and algorithm-accountability requirements. You do not need the details, only that AI must follow them.
- **Challenges of regulated workloads:** proving fairness, explaining decisions, controlling where data lives, and keeping audit trails.

Governance strategies

- **Policies:** acceptable-use rules, data-handling and retention policies, model approval processes.
- **Review cadence:** regular audits, reviews of model performance, and updates when regulations change.
- **Roles and training:** clear owners, cross-functional review boards, and team training on responsible AI.
- **Transparency standards and documentation:** Model Cards, source citations, dataset catalogues, data lineage.
- **Data governance:** data lifecycle management, logging, residency, monitoring and observation, retention.

EXAM TRAP Domain 5 traps

- "Who invoked the model?" = CloudTrail. "Is the config compliant?" = Config. "Give me the auditor's SOC report" = Artifact. "Collect audit evidence" = Audit Manager.
- "Find PII in S3" = Macie. "Keep traffic private" = PrivateLink / VPC endpoint. "Encrypt with my keys" = KMS.
- Under shared responsibility **you** configure IAM, encryption and data use; AWS does not do that for you.
- Least privilege is nearly always the right IAM answer.

DO THIS Recall drill

- Split a Bedrock app's security duties between AWS and you.
- Which service: audit trail, compliance reports, PII in S3, private network path, key management?
- Order the Security Scoping Matrix scopes from least to most responsibility.
- List four governance strategies an organisation should adopt for AI.

Look-alike services and concepts: compare and contrast

Exam writers love pairs that sound alike. For each pair, learn the **one sentence that separates them**.

Pair / group	How to tell them apart
Bedrock vs SageMaker	Bedrock = use ready-made foundation models through an API (managed, serverless). SageMaker = build, train, tune and host your own models with full control.
Rekognition vs Textract	Rekognition understands <i>images and video</i> (objects, faces, unsafe content). Textract pulls <i>text, forms and tables out of documents</i> .
Comprehend vs Kendra vs Lex	Comprehend analyses text (sentiment, entities, PII). Kendra searches your documents. Lex builds conversational bots.
Transcribe vs Polly vs Translate	Transcribe: speech to text. Polly: text to speech. Translate: text in one language to another.
Guardrails vs Clarify vs A2I	Guardrails filter GenAI inputs and outputs at run time. Clarify detects bias and explains predictions. A2I sends predictions to human reviewers.
Ground Truth vs A2I	Ground Truth = humans <i>label training data</i> . A2I = humans <i>review live predictions</i> .
RAG vs fine-tuning vs continued pre-training	RAG adds knowledge at query time (no model change). Fine-tuning teaches a task or style with labeled pairs. Continued pre-training teaches a domain with unlabeled text.
Knowledge Bases vs Agents	Knowledge Bases retrieve information (RAG). Agents plan steps and take actions by calling APIs; they can use knowledge bases too.
Amazon Q Business vs Q Developer vs PartyRock	Q Business answers from company data. Q Developer helps write and understand code and AWS. PartyRock is a no-code playground for experiments.
CloudTrail vs CloudWatch vs Config	CloudTrail = who did what (API audit). CloudWatch = metrics, logs, alarms (performance). Config = resource configuration history and compliance.
Macie vs Inspector	Macie finds sensitive data in S3. Inspector scans workloads for software vulnerabilities.
Artifact vs Audit Manager	Artifact = download AWS's own compliance reports. Audit Manager = collect evidence about <i>your</i> usage for audits.
Precision vs recall	Precision: of my positive calls, how many were right (avoid false alarms). Recall: of all true cases, how many did I find (avoid misses).
Transparency vs explainability	Transparency: openness about how the system is built and used. Explainability: reasons for one specific output.
Overfitting vs underfitting	Overfit: memorised training data, fails on new data. Underfit: too simple, fails on both.
On-demand vs Batch vs Provisioned Throughput	On-demand: pay per token, flexible. Batch: cheaper, offline jobs. Provisioned: reserved capacity for steady traffic and for custom models.
Model Cards vs Model Monitor	Model Cards document a model (governance). Model Monitor watches a deployed model for drift.
Data drift vs concept drift	Data drift: inputs change. Concept drift: the meaning of the right answer changes.

Master service picker and keyword decoder

If the scenario says... choose...

Scenario	Choose
Extract text, forms and tables from scanned documents	Amazon Textract
Detect objects, faces, unsafe content in images or video	Amazon Rekognition
Sentiment, entities, key phrases, PII in text	Amazon Comprehend
Convert speech to text / text to speech	Transcribe / Polly
Translate between languages	Amazon Translate
Build a chatbot or voice bot	Amazon Lex
Enterprise document search	Amazon Kendra (or Amazon Q Business)
Personalised recommendations	Amazon Personalize
Use foundation models by API, no infrastructure	Amazon Bedrock
Build, train and deploy a custom ML model	Amazon SageMaker
Analyst builds a model with no code	SageMaker Canvas
Automatically try algorithms and choose the best	SageMaker Autopilot
Start from a pre-trained or open model in SageMaker	SageMaker JumpStart
Prepare and transform data visually	SageMaker Data Wrangler
Label training data with humans	SageMaker Ground Truth
Answers grounded in company documents, no retraining	RAG with Bedrock Knowledge Bases
FM must call APIs and complete multi-step tasks	Agents for Amazon Bedrock
Block topics, filter toxic text, redact PII in GenAI	Guardrails for Amazon Bedrock
Compare models on quality with human or automatic scoring	Bedrock model evaluation
Detect bias or explain predictions	SageMaker Clarify
Route low-confidence predictions to humans	Amazon Augmented AI (A2I)
Detect drift in production	SageMaker Model Monitor
Document a model for governance	SageMaker Model Cards
Employees ask questions of internal data	Amazon Q Business
Developer assistance and code generation	Amazon Q Developer
No-code GenAI experiments	PartyRock / Bedrock playground
Audit trail of API calls	AWS CloudTrail
Discover PII in S3	Amazon Macie
Download compliance reports	AWS Artifact
Private connectivity to Bedrock	VPC endpoint (AWS PrivateLink)
Manage encryption keys	AWS KMS

Keyword decoder: what the question wording is hinting at

Phrase in the question	Usually points to
least operational overhead, fully managed, serverless	Bedrock, pre-built AI services, managed features
no ML expertise, fastest to implement	Pre-built AI services, Canvas, Bedrock
most cost-effective, lowest cost first step	Prompt engineering, on-demand or batch, smaller model, pre-built service
up-to-date, internal documents, without retraining	RAG (Knowledge Bases)
reduce hallucinations	RAG, lower temperature, grounding checks, clear prompts
consistent, deterministic answers / more creative	Lower temperature / higher temperature
labeled data vs unlabeled corpus	Fine-tuning vs continued pre-training
explain, feature importance, bias	SageMaker Clarify
human review, human in the loop	Amazon A2I
multi-step task, call APIs, take action	Bedrock Agents
block harmful content, mask PII in responses	Bedrock Guardrails
who called, audit	CloudTrail
least privilege, permissions	IAM roles and policies
model quality declines over time	Model Monitor, retrain (drift)
compliance report for auditors	AWS Artifact
custom model must be served on Bedrock	Provisioned Throughput

One-page cheat sheet (read this before the exam)

Exam facts	Ideas to remember
65 questions (50 scored, 15 unscored) 700 / 1000 to pass No penalty for guessing Weights: D1 20%, D2 24%, D3 28%, D4 14%, D5 14%	Highest layer that solves the problem wins Managed and simple beats custom and complex Prompt engineering, then RAG, then fine-tuning, then continued pre-training
Formulas $\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$ $\text{Recall} = \text{TP}/(\text{TP}+\text{FN})$ $\text{F1} = \text{balance of both}$ Accuracy lies on imbalanced data	Text metrics ROUGE = summaries BLEU = translation BERTScore = meaning Perplexity: lower is better
Learning types Supervised = labeled Unsupervised = unlabeled (clustering) Reinforcement = rewards Self-supervised = how FMs pre-train	Fit Overfit = great train, bad test (more data, regularise, simplify) Underfit = bad on both (more complexity, features)
LLM controls Low temperature = consistent, factual High temperature = creative Top P / Top K limit candidates Context window = max tokens per request	RAG flow Chunk, embed, store vectors, embed query, retrieve, add to prompt, generate Needs embeddings model plus vector DB
Bedrock Knowledge Bases (RAG), Agents (actions), Guardrails (safety), Evaluation, Customisation On-demand, Batch, Provisioned Throughput (needed for custom models)	Responsible AI Fairness, explainability, privacy and security, safety, controllability, veracity and robustness, governance, transparency
Security You: IAM, data, encryption use AWS: infrastructure KMS = keys, PrivateLink = private path, CloudTrail = audit, Macie = PII in S3, Artifact = reports, Config = compliance state	Scoping matrix (least to most ownership) Consumer app, enterprise app, pre-trained model, fine-tuned model, self-trained model

Practice questions (30)

Set a 45-minute timer, answer without notes, then check the answer key on the next pages.

Questions 1 to 6 are Domain 1, 7 to 12 Domain 2, 13 to 20 Domain 3, 21 to 25 Domain 4, 26 to 30 Domain 5. Circle any you guessed; treat guesses as misses.

1. A retailer wants to group customers into segments but has no predefined labels. Which approach fits?

- A. Supervised classification
- B. Unsupervised clustering
- C. Reinforcement learning
- D. Linear regression

2. A model scores 98% on training data but 70% on test data. What is the best description?

- A. Overfitting
- B. Underfitting
- C. Concept drift
- D. High bias

3. A hospital screens for a rare disease. Missing a sick patient is far worse than a false alarm. Which metric matters most?

- A. Precision
- B. BLEU
- C. RMSE
- D. Recall

4. Which AWS service converts recorded customer calls into text?

- A. Amazon Lex
- B. Amazon Comprehend
- C. Amazon Polly
- D. Amazon Transcribe

5. A team with no ML experience wants to predict customer churn from spreadsheet data through a visual interface. What should they use?

- A. SageMaker Ground Truth
- B. AWS Inferentia
- C. SageMaker Canvas
- D. Amazon Rekognition

6. After deployment, prediction quality drops because customer behaviour changed. Which capability detects this?

- A. SageMaker JumpStart
- B. SageMaker Ground Truth
- C. SageMaker Data Wrangler
- D. SageMaker Model Monitor

7. A developer needs consistent, factual answers from a policy Q&A assistant. Which inference setting should be adjusted?

- A. Lower the temperature
- B. Raise the temperature
- C. Increase max tokens
- D. Add more stop sequences

8. What is an embedding?

- A. A numeric vector representing the meaning of data
- B. A fine-tuning dataset
- C. A reusable prompt template
- D. A compressed copy of a model

9. A model confidently cites a regulation that does not exist. This behaviour is called:

- A. Hallucination
- B. Overfitting
- C. Tokenisation
- D. Drift

10. Which architecture underlies most modern large language models?

- A. k-means
- B. Convolutional network
- C. Decision tree
- D. Transformer

11. A marketing intern wants to prototype a generative AI app with no code. What is the best fit?

- A. PartyRock
- B. Amazon Macie
- C. AWS Trainium
- D. AWS Glue

12. A user pastes a very long document and the model cannot process all of it. Which limit is being hit?

- A. Provisioned Throughput
- B. Temperature
- C. Top K
- D. Context window

13. A support chatbot must answer from the latest HR policies without retraining the model. Which technique fits?

- A. Raising the temperature
- B. Fine-tuning
- C. Retrieval-Augmented Generation (RAG)
- D. Pre-training from scratch

14. In a RAG design on AWS, which is a suitable place to store embeddings?

- A. Amazon CloudFront
- B. AWS Glue DataBrew
- C. Amazon Athena
- D. Amazon OpenSearch Service (vector engine)

15. A law firm has a huge unlabeled corpus of legal text and wants the model to learn legal vocabulary. Which approach?

- A. Continued pre-training
- B. Prompt engineering
- C. Fine-tuning with labeled pairs
- D. RAG only

16. What is the cheapest, fastest first step to improve the quality of summaries from an FM?

- A. Buy Provisioned Throughput
- B. Train a new model
- C. Fine-tune the model
- D. Improve the prompt

17. A prompt contains three example inputs with the desired outputs. Which technique is this?

- A. Few-shot prompting
- B. Negative prompting
- C. Zero-shot prompting
- D. Chain-of-thought

18. An application must plan multi-step tasks such as checking an order and issuing a refund by calling company APIs. Which feature?

- A. Amazon Kendra
- B. Amazon Textract
- C. Guardrails for Amazon Bedrock
- D. Agents for Amazon Bedrock

19. Which metric commonly evaluates summarisation by comparing overlap with a reference summary?

- A. RMSE
- B. BLEU
- C. ROUGE
- D. AUC

20. A company fine-tuned a model on Amazon Bedrock. What is required to serve it?

- A. Provisioned Throughput
- B. Nothing extra
- C. Batch inference only
- D. On-demand pricing only

21. A bank must check whether its loan model treats groups differently and explain which features drive decisions. Which service?

- A. AWS CloudTrail
- B. Amazon Transcribe
- C. SageMaker Clarify
- D. Amazon Macie

22. A company wants a Bedrock app to refuse investment-advice questions and mask personal data in responses. Which feature?

- A. Guardrails for Amazon Bedrock
- B. SageMaker Model Cards
- C. Amazon A2I
- D. AWS Config

23. Which statement about explainability is correct?

- A. Simpler models are often easier to interpret but may be less accurate
- B. Explainability is never required
- C. Complex models are always easier to explain
- D. Explainability and performance never conflict

24. Low-confidence document extraction results must be sent to human reviewers. Which service?

- A. Amazon Neptune
- B. Amazon Augmented AI (A2I)
- C. Amazon Comprehend
- D. Amazon Personalize

25. Which service records a model's intended use, training data, evaluation results and risks for governance?

- A. Amazon Lex
- B. AWS CloudTrail
- C. SageMaker Model Cards
- D. Amazon Kendra

26. Under the shared responsibility model, who configures IAM permissions and decides how data is encrypted and used for a Bedrock app?

- A. The customer
- B. The model provider
- C. AWS only
- D. No one

27. Which service provides an audit trail of who invoked Bedrock APIs?

- A. Amazon Inspector
- B. AWS CloudTrail
- C. AWS Artifact
- D. Amazon Macie

28. Before fine-tuning, you must confirm that the S3 training data contains no personal information. Which service helps discover PII?

- A. Amazon CloudWatch
- B. AWS Config
- C. AWS Trusted Advisor
- D. Amazon Macie

29. Traffic between a VPC and Amazon Bedrock must not traverse the public internet. What should you use?

- A. A VPC interface endpoint (AWS PrivateLink)
- B. A public NAT only
- C. An internet gateway
- D. CloudFront

30. Auditors need AWS's SOC and ISO compliance reports. Where do you get them?

- A. AWS CloudTrail
- B. AWS Artifact
- C. Amazon Inspector
- D. SageMaker Clarify

Answer key with explanations

For each miss, find the topic in the guide and re-read its **Why** and **How** notes. Your goal: every missed question becomes a correct one on the second try.

#	Answer	Why (domain)
1	B	Grouping unlabeled data by similarity is clustering, which is unsupervised. Classification and regression need labels; reinforcement learning needs a reward signal. (Domain 1)
2	A	Great on training and poor on new data means the model memorised the training set (overfitting, high variance). (Domain 1)
3	D	Recall measures how many real cases are caught (few false negatives). RMSE is for regression and BLEU is for translation. (Domain 1)
4	D	Transcribe is speech to text. Polly is the reverse, Lex builds bots, and Comprehend analyses text you already have. (Domain 1)
5	C	Canvas provides no-code ML for analysts. Ground Truth labels data, Rekognition analyses images, Inferentia is a chip. (Domain 1)
6	D	Model Monitor watches production data and predictions for drift and quality problems. (Domain 1)
7	A	Lower temperature reduces randomness so answers are more repeatable and factual. (Domain 2)
8	A	Embeddings map text or images to vectors so similar meanings are close together, enabling semantic search and RAG. (Domain 2)
9	A	Confident but fabricated output is a hallucination. RAG and Guardrails grounding checks reduce it. (Domain 2)
10	D	Transformers use self-attention to relate tokens and power today's LLMs. (Domain 2)
11	A	PartyRock is a no-code playground built on Bedrock for experiments. (Domain 2)
12	D	The context window is the maximum tokens (input plus output) the model can handle in one request. (Domain 2)
13	C	RAG retrieves current documents at query time and adds them to the prompt; no model change needed. (Domain 3)
14	D	OpenSearch supports vector search. Athena queries S3 with SQL, DataBrew prepares data, and CloudFront is a CDN. (Domain 3)
15	A	Continued pre-training uses large unlabeled domain text. Fine-tuning needs labeled prompt and response pairs. (Domain 3)
16	D	Prompt engineering needs no training or infrastructure and is instant to iterate. (Domain 3)
17	A	Including a few worked examples is few-shot prompting. (Domain 3)
18	D	Agents let an FM reason through steps and call APIs (action groups). (Domain 3)
19	C	ROUGE measures n-gram overlap and is the standard summarisation metric. BLEU is for translation. (Domain 3)
20	A	Custom models on Bedrock are served with Provisioned Throughput. (Domain 3)
21	C	Clarify detects bias and provides feature-attribution explanations. (Domain 4)
22	A	Guardrails provide denied topics and PII redaction. (Domain 4)
23	A	There is usually a trade-off between model performance and interpretability. (Domain 4)
24	B	A2I adds human review workflows to ML predictions. (Domain 4)
25	C	Model Cards are structured documentation for governance and transparency. (Domain 4)

#	Answer	Why (domain)
26	A	AWS secures the infrastructure; the customer manages access, data, and how the service is used. (Domain 5)
27	B	CloudTrail logs API calls for auditing. Macie finds sensitive data; Artifact gives compliance reports. (Domain 5)
28	D	Macie uses ML to discover sensitive data in S3. (Domain 5)
29	A	PrivateLink interface endpoints keep traffic on the AWS network. (Domain 5)
30	B	AWS Artifact provides on-demand access to AWS compliance reports and agreements. (Domain 5)

REMEMBER Score guide

- 27 to 30 correct: strong. Skim the traps once more and rest.
- 22 to 26: close. Re-read the domains where you missed the most, then redo those questions.
- Below 22: do a second pass on Domains 3 and 2 first (52% of the exam), then retake this set tomorrow.

Glossary and flashcards

Read the term, cover the definition, say it aloud, then check.

Term	Definition
Agent (Bedrock)	FM that plans multi-step tasks and calls APIs and knowledge bases.
Algorithm	The procedure a model uses to learn from data.
Bias (data or model)	Systematic unfairness or error; in ML also error from an overly simple model.
Chain-of-thought	Prompting the model to reason step by step.
Chunking	Splitting documents into pieces before embedding for RAG.
Classification	Predicting a category (supervised).
Clustering	Grouping similar items without labels (unsupervised).
Context window	Maximum tokens a model can process in one request.
Continued pre-training	Further training on large unlabeled domain data.
Data drift	Production input data changes from what the model was trained on.
Deep learning	ML with multi-layer neural networks.
Distillation	Training a small model to imitate a larger one.
Embedding	Vector of numbers capturing meaning.
Epoch	One full pass through the training data.
Feature	An input variable to a model.
Fine-tuning	Adjusting a pre-trained model with labeled examples for a task or style.
Foundation model	Large, broadly pre-trained model adaptable to many tasks.
Guardrails	Configurable safety filters on GenAI inputs and outputs.
Hallucination	Confident but false model output.
Hyperparameter	A setting chosen before training (learning rate, epochs).
Inference	Using a trained model to make predictions.
Jailbreak	Prompt that tricks a model into bypassing safety rules.
Label	The correct answer attached to a training example.
LLM	Large language model; predicts the next token.
MLOps	Practices to automate and monitor the ML lifecycle.
Multimodal	Handles multiple data types such as text and images.
Overfitting	Model memorises training data and fails on new data.
Perplexity	Measure of how well a language model predicts text; lower is better.
Prompt injection	Malicious input that overrides your instructions.
Provisioned Throughput	Reserved Bedrock model capacity; needed for custom models.
RAG	Retrieve relevant data and add it to the prompt before generating.
Regression	Predicting a number (supervised).
Reinforcement learning	Learning by rewards and penalties.

Term	Definition
RLHF	Reinforcement learning from human feedback.
Self-attention	Transformer mechanism relating each token to others.
Temperature	Controls randomness of output.
Token	Chunk of text a model reads or writes; the unit of pricing.
Top K / Top P	Limits on which candidate tokens can be sampled.
Transfer learning	Reusing a trained model for a new task.
Underfitting	Model too simple; poor on training and new data.
Vector database	Stores embeddings and finds nearest matches.
Zero-shot / few-shot	Prompting with no examples / with a few examples.

Exam-day playbook

The night before and the morning of

- Sleep. A rested brain beats one more hour of cramming.
- Re-read only: the cheat sheet, the look-alike table, and your missed practice questions.
- Confirm your ID, exam time, and (if online) your room and computer check. Arrive or log in early.
- Eat, hydrate, and bring nothing you cannot use. Assume no notes.

How to attack each question

1. **Read the last sentence first** to see what is asked: "most cost-effective", "least operational overhead", "most secure".
2. **Read the scenario** and underline the requirement words (real-time, no retraining, no ML expertise, regulated data).
3. **Eliminate** the two clearly wrong options. Then choose between the remaining pair using the keyword decoder.
4. **Prefer managed, simple, least-privilege** answers when two options both work.
5. For **multiple response**, select exactly the number requested. For **ordering**, mentally replay the process (RAG flow, ML lifecycle). For **matching**, do the sure ones first.
6. **Flag and move on** if stuck; do not spend more than about a minute and a half on any one item. Answer everything: there is no penalty for guessing.

Time plan

- Divide your total exam minutes by 65 to get your average time per question (confirm the duration on your booking).
- First pass: answer everything you are sure of, flag the rest. Second pass: return to flagged questions. Keep about 10 minutes at the end for a quick review.
- Up to 15 questions are unscored and unmarked. If a question looks bizarre or off-topic, do not panic; answer your best guess and move on.

REMEMBER If your mind goes blank

Ask: which layer of the stack? (pre-built AI service, Bedrock, SageMaker.) Which lifecycle stage? Which responsibility dimension? Those three questions unlock most items.

DO THIS Final 5-minute mental checklist

- Prompt engineering, then RAG, then fine-tuning, then continued pre-training.
- Labeled means fine-tuning; unlabeled means continued pre-training.
- Low temperature for consistency. High for creativity.
- Clarify for bias, Guardrails for content, A2I for humans, Model Cards for docs.
- CloudTrail audit, Macie PII, Artifact reports, PrivateLink private, KMS keys, IAM least privilege.

Based on the official AWS Certified AI Practitioner (AIF-C01) exam guide domains and weightings. AWS updates services and the exam over time, so confirm current details at aws.amazon.com/certification before you book. This guide is a study aid, not an official AWS publication.